Reporte del POC

Examen parcial – Métodos de Gran Escala

Ana P. Paredes Rodríguez

Problema del negocio

El cliente 1C Company está interesado en adquirir un producto de datos que prediga la demanda de sus items (haciendo distinción entre tienda y producto). Es un reto porque el producto de datos será clave para la toma de decisiones del consejo directivo y además se obtuvo un presupuesto para un producto mínimo viable (que es la versión más simple y funcional de un nuevo producto).

Después de recopilar las principales peticiones de la mesa directiva, se buscó que el producto de datos resolviera los siguientes intereses:

1. **Que no dependa de un notebook**: el resultado final se despliega en una URL pública y cualquier persona puede tener acceso a los resultados de las predicciones.
2. **Que se pueda filtrar:** Las predicciones se pueden tener especificando el producto, la tienda y la categoría del producto. Esto para tener la información más específica posible.
3. **Tener las predicciones accesibles:** Se pueden descargar los archivos CSV de las predicciones por tienda o para todo el catálogo.
4. **Que se puedan hacer consultas:** Las predicciones están disponibles en un catalogo GLUE y conectadas a Athena. De modo que si el equipo necesita, se pueden hacer consultas específicas.
5. **Utiliza su proveedor de cloud:** Todos los servicios utilizados son de AWS por lo que no es necesario contratar algún proveedor adicional. Además de que se decidió correr el modelo de forma local, lo que redujo el costo al no necesitar utilizar alguna plataforma para entrenar y correr el pipeline completo.
6. **Seguimiento del modelo:** Dentro del código del modelo se tienen logs y estos se guardan automáticamente, de forma que se tiene el registro de todos los logs generados al correr el modelo.
7. **Que no sea muy complicado:** El diagrama de flujo es sencillo y se buscó que se cumplieran con los requisitos sin caer en servicios muy complejos o en

procesos demasiado complicados. Además, se optó por solamente usar un modelo sencillo, que diera buenas predicciones y mantuviera la parsimonia.

8. **Tener un modelo confiable:** Al entrenar el modelo se obtuvo un RMSE de 0.7442 (al registrarlo a Kaggle se obtuvo un score de 1.02, un resultado bastante confiable).
9. **Que capture el feedback:** El producto de datos puede guardar el feedback e incluso se puede hacer un feedback específico por item-producto, de forma que se pueda saber qué predicción se tiene que revisar. Además, se guarda el nombre del usuario que dejó el feedback, así como la fecha y hora.

Arquitectura de la solución

Para la solución se optó por el siguiente diagrama de flujo:

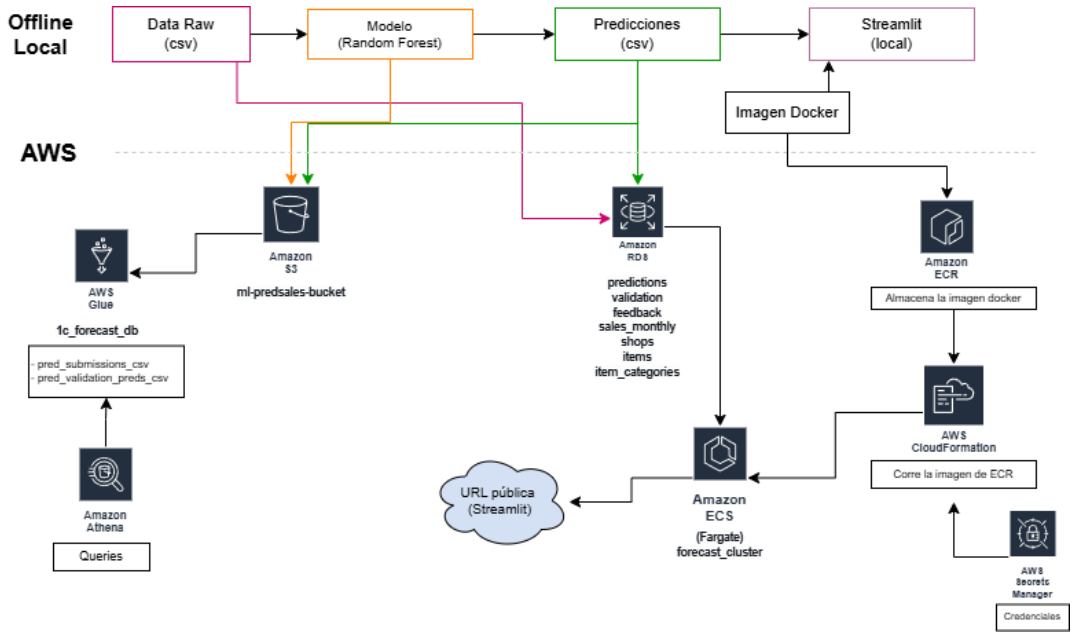


Ilustración 1. Diagrama de flujo del producto de datos

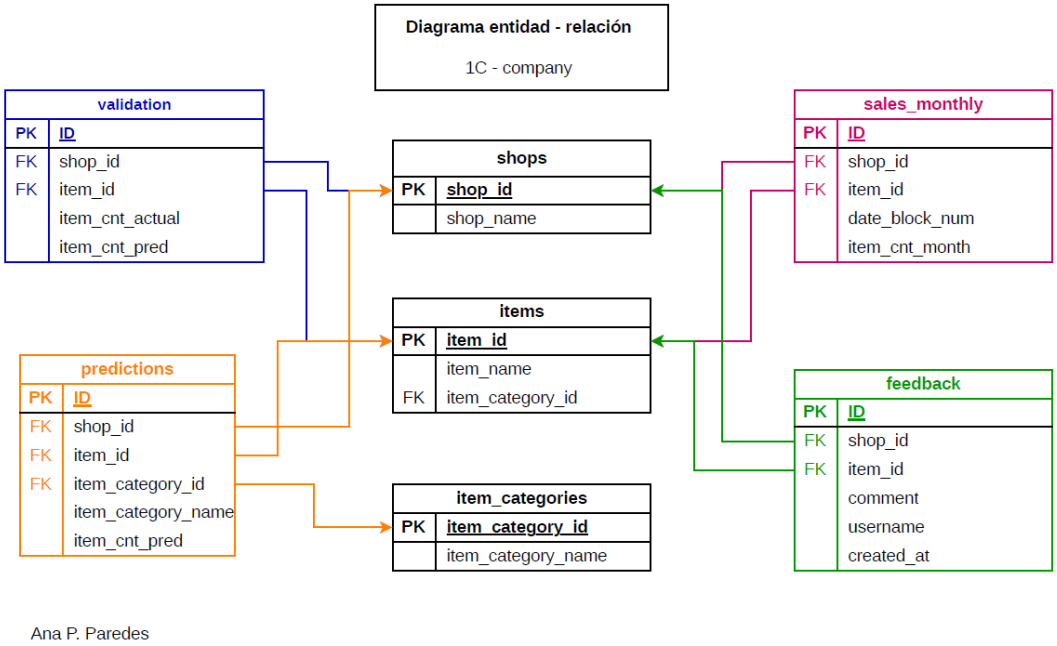
Se optó por correr el pipeline completo del modelo (de las tareas anteriores) de manera local, esto para reducir costos, aprovechar que se tenían todos los recursos disponibles y que sería más sencillo y ahorraría tiempo (clave en esta entrega).

Posteriormente se utilizaron esos datos para crear la app que despliega el Streamlit (de forma local), primero se hizo la estructura del streamlit con datos ficticios para definir cómo se vería y que haría falta. Posteriormente se incluyeron los datos de las predicciones obtenidas. Cabe mencionar que para desplegar el streamlit, se necesito crear una imagen Docker, una vez que se aprobó el diseño del streamlit, se subió la imagen docker a ECR.

Una vez que la imagen docker estaba almacenada en ECR, se corrió la imagen en un stack creado en CloudFormation (utilizando el YAML que vimos en clase) e incorporando los datos personales a través de Secrets Manager. De esta forma evitamos que se filtran contraseñas o información sensible. Una vez creado el stack y con la imagen Docker, se conecta a ECS para que utilicé la imagen de ECR y la despliegue en una URL pública. Cabe mencionar que se subieron 7 tablas a RDS para que se pudieran utilizar en ECS.

Paralelamente, tanto el modelo como las predicciones se guardaron en un bucker de S3, este bucket se conectó a través de crawler con glue. Se utilizó ese servicio porque era más rápido (de nuevo, el tiempo es vital). Una vez que las predicciones están en el catálogo GLUE, se conectó a Athena para poder hacer las queries.

Modelo de datos



El ERD muestra las tablas que se crearon y almacenaron en RDS, las 7 tablas son:

1. **Shops:** viene de los datos crudos y contiene el nombre de la tienda, se utilizó para poder crear el catálogo (una de las pestañas del streamlit)

2. **Items:** viene de los datos crudos y contiene el nombre del item y su categoría, también se utilizó para crear el catálogo
3. **Item_categories:** viene de los datos crudos y contiene el nombre de la categoría
4. **Sales_monthly:** Esta tabla se creó al agrupar las ventas de forma mensual, contiene el id de la tienda, del itam, el id del mes-año y la cantidad vendida al mes
5. **Predictions:** En esta tabla se tienen las predicciones y contiene el id de la tienda, del item, la categoría (incluído el nombre de la categoría) y la cantidad predicha.
6. **Validation:** Es similar a la tabla de predictions, pero con los datos de validación. Es decir, con los valores reales del mes 33 y sus predicciones, se utilizó para poder hacer la comparativa y evaluar el modelo.
7. **Feedback:** Esta tabla va almacenando los feedbacks que recibe el streamlit, tiene el id de la tienda, del item, el comentario, el usuario y la fecha de cuando se creó.

Pipeline de datos y de ML

El pipeline de los datos ya estaba definido de las tareas previas, se había corrido el modelo y se había logrado que se completara el proceso. De forma que se retomó el modelo y se utilizaron los siguientes comandos para llevar a cabo todo el pipeline:

Preprocesamiento de los datos:

```
cd src/preprocessing
uv run python __main__.py \
  --raw-dir ../../data/raw \
  --prep-dir ../../data/prep \
  --inference-dir ../../data/inference
```

Con los siguientes resultados

```
2026-04-26 15:51:50,459 - prep - INFO - ==================================================
2026-04-26 15:51:50,460 - prep - INFO - Starting data preparation pipeline (prep.py)
2026-04-26 15:51:50,460 - prep - INFO - ==================================================
2026-04-26 15:51:50,460 - prep - INFO - Loading sales data...
2026-04-26 15:51:53,849 - prep - INFO - Sales data loaded: 2,935,849 records
2026-04-26 15:51:53,937 - prep - INFO - Loading items data...
deloML-predsales/src/__pycache__ - prep - INFO - Items data loaded: 22,170 records
2026-04-26 15:51:54,045 - prep - INFO - Cleaning sales data...
2026-04-26 15:51:55,311 - prep - INFO - Removed 7,357 invalid records — 2,928,492 remaining
2026-04-26 15:51:55,311 - prep - INFO - Aggregating sales to monthly level...
2026-04-26 15:51:56,086 - prep - INFO - Monthly aggregation complete: 1,608,226 records — clip applied at 20
2026-04-26 15:51:56,087 - prep - INFO - Adding item category information...
2026-04-26 15:51:56,228 - prep - INFO - Building complete shop-item-month grid...
2026-04-26 15:51:56,620 - prep - INFO - Grid dimensions: 34 months x 424,098 active combinations
2026-04-26 15:52:07,309 - prep - INFO - Grid built: 14,419,332 total records
2026-04-26 15:52:10,921 - prep - INFO - Generating lag features: [1, 3, 6, 12]...
2026-04-26 15:52:16,575 - prep - INFO - Lag features generated successfully
2026-04-26 15:52:17,521 - prep - INFO - Filtered incomplete cases: 5,089,176 rows dropped — 9,330,156 remaining
2026-04-26 15:53:02,422 - prep - INFO - Prepared dataset saved: 9,330,156 records → grid_model.csv
2026-04-26 15:53:02,423 - prep - INFO - Data preparation completed in 71.96 seconds
2026-04-26 15:53:02,423 - prep - INFO - ==================================================
```

Se creó el grid_model con todas las combinaciones activas (9,330,156 observaciones) en tan solo 71.96 segundos.

```
2026-04-26 15:53:02,492 - prep_inference - INFO - ==================================================
2026-04-26 15:53:02,492 - prep_inference - INFO - Starting inference data preparation pipeline (prep_inference.py)
2026-04-26 15:53:02,492 - prep_inference - INFO - ==================================================
2026-04-26 15:53:02,492 - prep_inference - INFO - Loading test data...
2026-04-26 15:53:02,625 - prep_inference - INFO - Test data loaded: 214,200 records
2026-04-26 15:53:02,625 - prep_inference - INFO - Loading prepared grid model...
2026-04-26 15:53:10,591 - prep_inference - INFO - Grid model loaded: 9,330,156 records
2026-04-26 15:53:10,591 - prep_inference - INFO - Merging lag features into test data...
2026-04-26 15:53:10,733 - prep_inference - WARNING - lag_1: 102,796 records without historical data — will be filled w
ith 0
2026-04-26 15:53:10,876 - prep_inference - WARNING - lag_3: 102,796 records without historical data — will be filled w
ith 0
2026-04-26 15:53:11,018 - prep_inference - WARNING - lag_6: 102,796 records without historical data — will be filled w
ith 0
2026-04-26 15:53:11,171 - prep_inference - WARNING - lag_12: 102,796 records without historical data — will be filled
with 0
2026-04-26 15:53:11,182 - prep_inference - INFO - All lag features merged and null values filled with 0
2026-04-26 15:53:12,112 - prep_inference - INFO - Test with lags saved: 214,200 records → test_with_lags.csv
2026-04-26 15:53:12,113 - prep_inference - INFO - Inference preparation completed in 9.62 seconds
2026-04-26 15:53:12,113 - prep_inference - INFO - ==================================================
```

Se realizó el preprocesamiento para los datos de inferencia test_with_lags (214,200 observaciones) en tan solo 9.62 segundos.

Entrenamiento del modelo:

```
cd src/training
uv run python __main__.py \
  --prep-dir ../../data/prep \
  --artifacts-dir ../../artifacts \
  --no-random-search
```

```
2026-04-26 16:34:30,133 - train - INFO - ==================================================
2026-04-26 16:34:30,134 - train - INFO - Starting training pipeline (train.py)
2026-04-26 16:34:30,134 - train - INFO - ==================================================
2026-04-26 16:34:30,134 - train - INFO - Loading prepared dataset...
2026-04-26 16:34:39,973 - train - INFO - Prepared dataset loaded: 9,330,156 records
2026-04-26 16:34:43,537 - train - INFO - Train set: 8,906,058 records (months 0-32)
2026-04-26 16:34:43,537 - train - INFO - Validation set: 424,098 records (month 33)
2026-04-26 16:34:43,624 - train - INFO - Training Random Forest — n_estimators=100, max_depth=10, random_state=10
2026-04-26 16:36:27,694 - train - INFO - Model training complete
2026-04-26 16:36:28,027 - train - INFO - Validation RMSE (Random Forest): 0.7442
2026-04-26 16:36:28,162 - train - INFO - Model saved: random_forest_lags.pkl
2026-04-26 16:36:28,162 - train - INFO - Training pipeline completed in 118.03 seconds
2026-04-26 16:36:28,162 - train - INFO - ==================================================
```

Se entrenó un modelo random forest y se obtuvo el RMSE 0.7442. Adicionalmente, se guardó el modelo: random_forest_lags.pkl y todo se logró en 118.03 segundos.

Cabe mencionar que se tuvo que quitar el random search porque moría el proceso debido al tiempo de carga, lo mismo pasaba en las tareas en sagemaker. De modo que para lograr una respuesta, se optó por quitarlo.

Inferencia:

```
cd ../inference
uv run python __main__.py \
  --inference-dir ../../data/inference \
  --artifacts-dir ../../artifacts \
  --predictions-dir ../../data/predictions
```

```
2026-04-26 16:39:57,161 - inference - INFO - ==================================================
2026-04-26 16:39:57,161 - inference - INFO - Starting inference pipeline (inference.py)
2026-04-26 16:39:57,161 - inference - INFO - ==================================================
2026-04-26 16:39:57,162 - inference - INFO - Loading trained model...
2026-04-26 16:39:58,999 - inference - INFO - Model loaded successfully: random_forest_lags.pkl
2026-04-26 16:39:59,000 - inference - INFO - Loading inference data...
2026-04-26 16:39:59,272 - inference - INFO - Inference data loaded: 214,200 records
2026-04-26 16:39:59,273 - inference - INFO - Generating predictions...
2026-04-26 16:39:59,610 - inference - INFO - Predictions generated: 214,200 records
2026-04-26 16:40:00,360 - inference - INFO - Predictions saved: 214,200 records → submissions.csv
2026-04-26 16:40:00,360 - inference - INFO - Inference pipeline completed in 3.20 seconds
2026-04-26 16:40:00,361 - inference - INFO - ==================================================
```

Se obtuvieorn las predicciones (214,200 observaciones) en tan solo 3.20 segundos.

Evaluación del modelo

Como se había comentado, se corrió el modelo y se obtuvo un RSME de 0.7442. Sin embargo, para poder compararlo, se realizó un modelo baseline donde se predecía que la venta del siguiente mes iba a ser exactamente igual a la venta del mes anterior. Esto se hizo para asegurar que al menos se tiene un desempeño mejor que si hubieras hecho ese modelaje. De aquí se realizaron dos especificaciones: la primera cuando se tenían lags desordenados (es decir, la venta del mes siguiente es igual a la venta del

último mes que reportó alguna venta) y bajo este supuesto se obtuvo un RMSE de 2.78. Cuando se ordenaron los lags de tal forma que la venta del mes siguiente sí sea exactamente igual a la venta del mes anterior se obtuvo un RMSE de 0.8341. De modo que el modelo ajusta mejor que dos especificaciones del modelo baseline.

A su vez, al realizar las gráficas de evaluación vs ground truth, se observó que variaba dependiendo del producto y la tienda.

Ilustración 2. Ejemplo

Así, también se incluyó el RMSE por categoría de producto

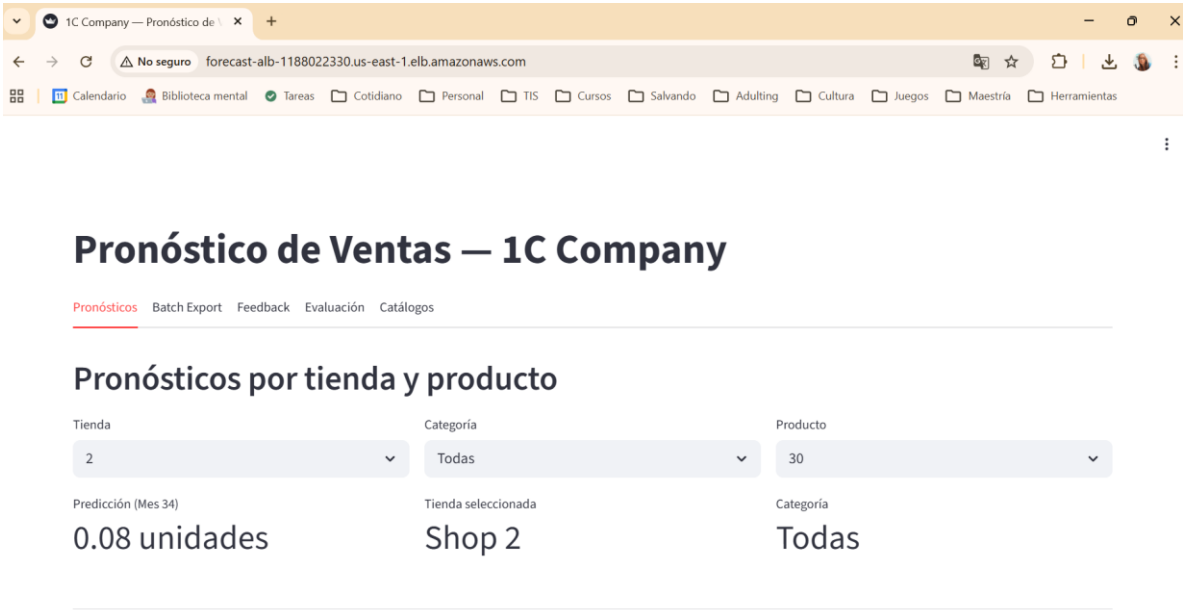
RMSE por categoría — Tienda completa

	item_category_name	item_category_id	rmse
17	Аксессуары - PS4	3	0.9792
18	Аксессуары - PS3	2	0.8633
19	Подарки - Мягкие игрушки	63	0.7804
20	Книги - Комиксы, манга	47	0.777
21	Аксессуары - XBOX ONE	7	0.767
22	Игры - PS3	19	0.6491
23	Подарки - Сувениры	69	0.6371
24	Музыка - CD локального производства	55	0.6118
25	Подарки - Фигурки	72	0.58
26	Подарки - Сувениры (в навеску)	70	0.5708

Tour de la aplicación

La URL pública es: http://forecast-alb-1188022330.us-east-1.elb.amazonaws.com/

Y se puede consultar en cualquier navegador, lo primero que saldrá será esta vista:



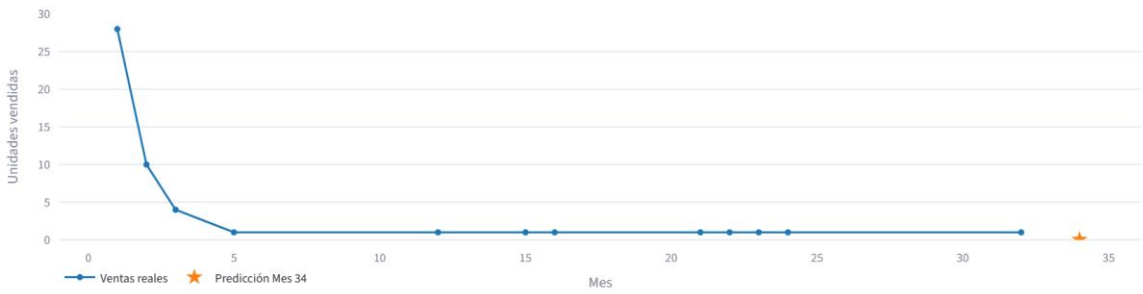
Como se puede ver, se tienen 5 pestañas:

1. Pronósticos: Las predicciones
2. Batch Export: La descarga de las predicciones
3. Feedback: La retroalimentación sobre las predicciones
4. Evaluación: La comparación entre las predicciones y los valores reales (use el mes 33 de referencia)
5. Catálogos: Los catálogos para tienda, categoría y producto id

Pronósticos

Se tiene la predicción por tienda y producto y se indica la categoría a la que corresponde el producto. Inclusive, si se selecciona una categoría, solamente aparecen las tiendas que lo venden o los productos que entran en esas categorías.

Histórico de ventas — Producto 30



El histórico muestra únicamente los meses en que el producto registró ventas en esta tienda. Los meses sin datos indican que el producto no tuvo actividad comercial en ese período.

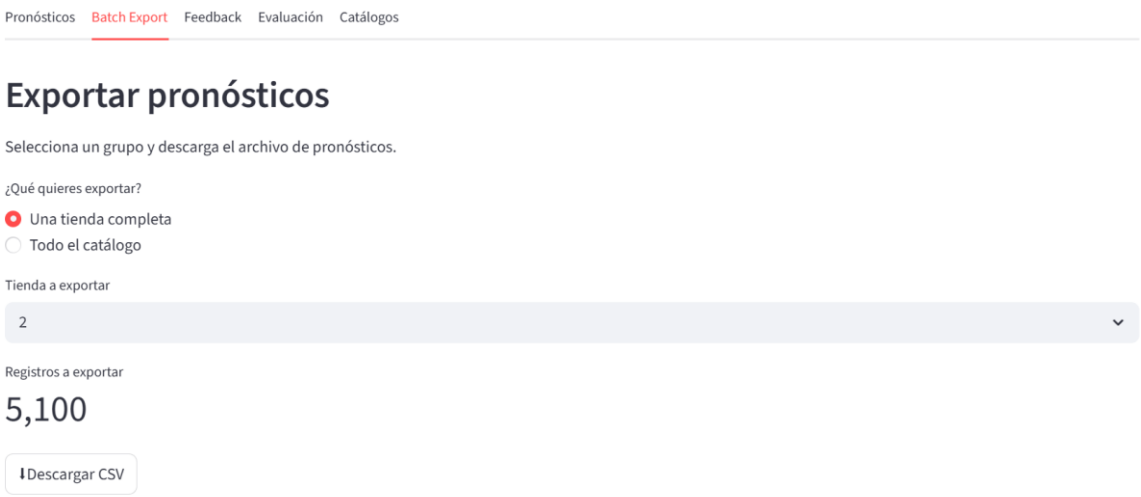
Se incluyó una gráfica del histórico de ventas de color azul y la estrellita naranja es la predicción realizada.

Todos los pronósticos — Tienda 59

	shop_id	item_id	item_category_name	item_cnt_pred
0	59	1876	Игры - PS3	1.2607
1	59	16790	Игры - PS4	0.4451
2	59	16560	Кино - DVD	0.0777
3	59	16825	Игры - PS4	0.4866
4	59	16988	Кино - DVD	0.0777
5	59	12646	Кино - DVD	0.0777
6	59	12362	Кино - DVD	0.522
7	59	5037	Игры - PS3	1.0407
8	59	5320	Музыка - CD локального производства	0.0777
9	59	5233	Игры - PS3	2.0064

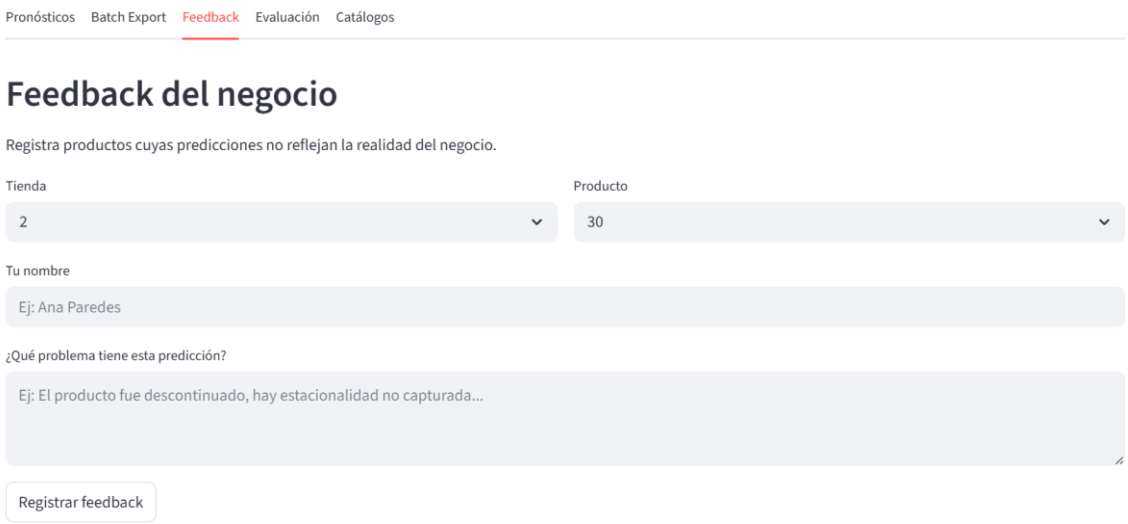
Y finalmente se tiene la tabla con todos las predicciones mencionando el id de la tienda, del itam, el nombre de la categoría y el pronóstico.

Batch export



En esta pestaña se pueden exportar las predicciones y puse la opción de exportar las predicciones para una tienda o todo el catálogo. Se puede indicar la tienda de interés y los registros que se van a exportar.

Feedback



En el feedback se pregunta por la tienda y producto, el nombre del usuario y el problema que se tiene con algunas opciones para dar ejemplos. Además del botón de registrar feedback.

Productos con problemas reportados

	shop_id	item_id	username	comment	created_at
0	2	30	Ana Paredes	Increíbles predicciones, gran diseño 10/10	2026-04-30 01:45:23

Adicionalmente, se tiene el registro de los problemas reportados, para poder tener un seguimiento.

Evaluación

Pronóstico de Ventas — 1C Company

Pronósticos Batch Export Feedback Evaluación Catálogos

Evaluación del modelo — Mes 33

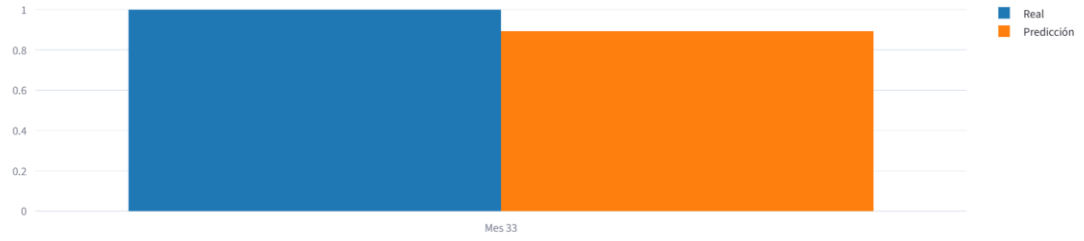
Comparación entre las ventas reales del mes 33 y las predicciones del modelo para ese mismo mes.

Tienda: 58

Producto: 10209

Ventas reales	Predicción	Error absoluto
1.00	0.89	0.11

En evaluación se tiene la comparación entre las ventas reales y las predicciones del mismo mes (mes 33). Se reportó el error absoluto para que sea más fácil ver la diferencia entre las predicciones.



Se incluye la comparación de forma gráfica.

Resumen por producto — Tienda 58

	shop_id	item_id	item_cnt_actual	item_cnt_pred	error_absoluto
80	58	961	0	0.0777	0.0777
81	58	962	0	0.1038	0.1038
82	58	963	0	0.0777	0.0777
83	58	964	0	0.1742	0.1742
84	58	965	0	0.0485	0.0485
85	58	968	0	0.6145	0.6145
86	58	969	0	0.0777	0.0777
87	58	970	0	0.0777	0.0777
88	58	971	0	0.0777	0.0777
89	58	972	0	0.1731	0.1731

También se puso una tabla con el resumen, incluyendo la comparación entre las actuales y las predicciones.

RMSE por categoría — Tienda completa

	item_category_name	item_category_id	rmse
26	Подарки - Сувениры (в навеску)	70	0.5708
27	Подарки - Настольные игры	64	0.5594
28	Игровые консоли - XBOX ONE	16	0.5308
29	Кино - Blu-Ray	37	0.5046
30	Кино - Коллекционное	41	0.4768
31	Программы - 1С:Предприятие 8	73	0.473
32	Игровые консоли - XBOX 360	15	0.4352
33	Подарки - Гаджеты, роботы, спорт	62	0.4266
34	Подарки - Развитие	67	0.4093
35	Игры - PSP	21	0.3785

Para complementar la evaluación, también se incluyó el cálculo de RMSE por categoría.

RMSE por tienda — Todas las tiendas

	shop_id	rmse
37	41	0.516
38	16	0.5138
39	10	0.4868
40	52	0.4865
41	56	0.4727
42	4	0.4546
43	44	0.4539
44	51	0.2475
45	27	0.1735
46	29	0.0864
47	54	0.081

Y también el cálculo de RMSE por tienda.

Catálogo

Pronósticos Batch Export Feedback Evaluación Catálogos

Catálogos de referencia

Tiendas Productos Categorías

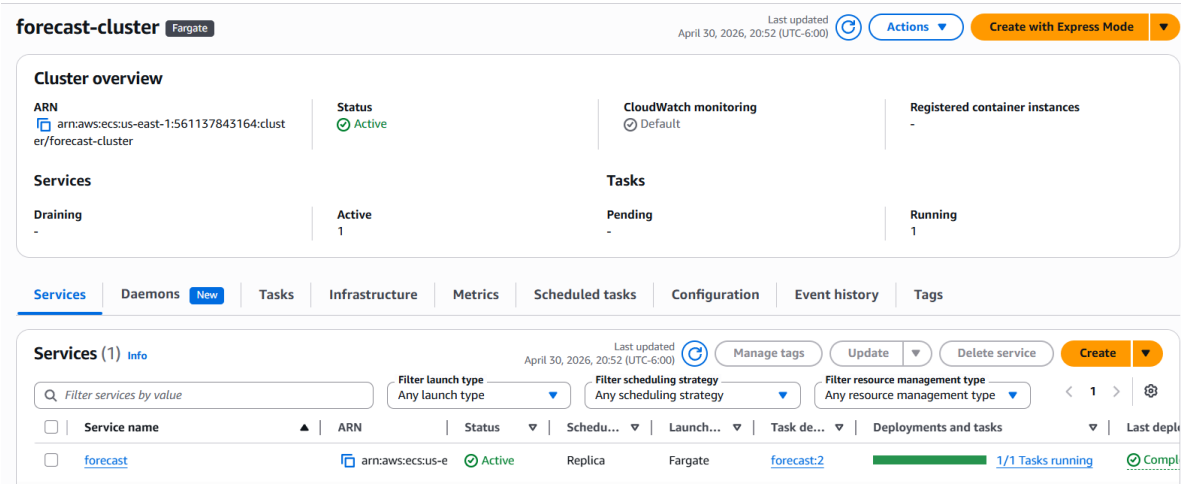
Tiendas

	shop_id	shop_name
0	0	!Якутск Орджоникидзе, 56 фран
1	1	!Якутск ТЦ "Центральный" фран
2	2	Адыгея ТЦ "Мега"
3	3	Балашиха ТРК "Октябрь-Киномир"
4	4	Волжский ТЦ "Волга Молл"
5	5	Вологда ТРЦ "Мармелад"
6	6	Воронеж (Плехановская, 13)

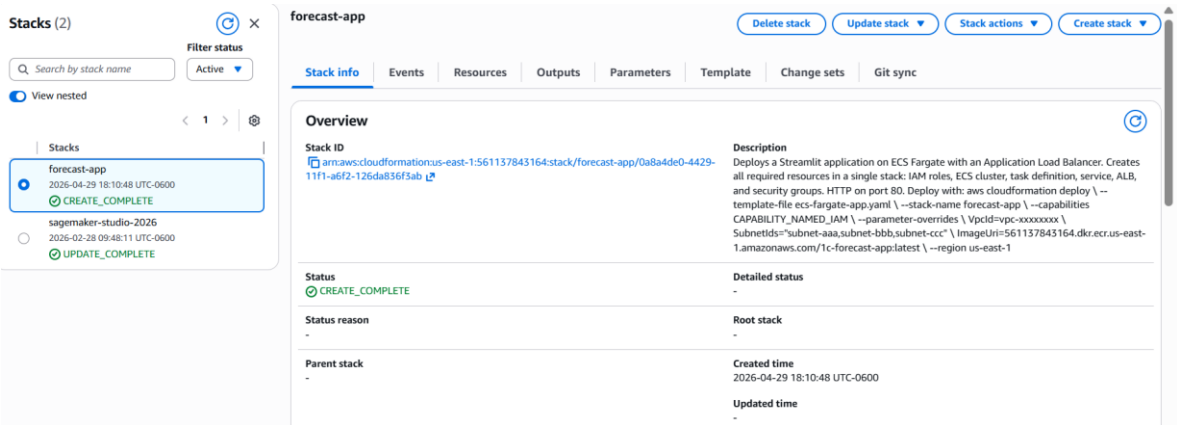
Esta pestaña es más auxiliar y sirve para poder tener los valores de cada id, así cuando el equipo tengan duda, tendran los datos a la mano.

Screenshots de los recursos de AWS desplegados

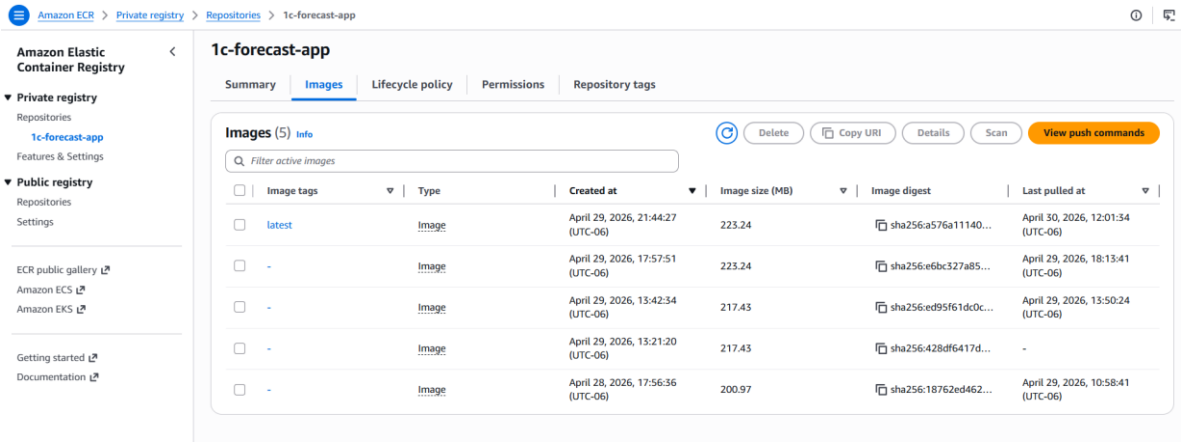
Consola de ECS con el servicio corriendo



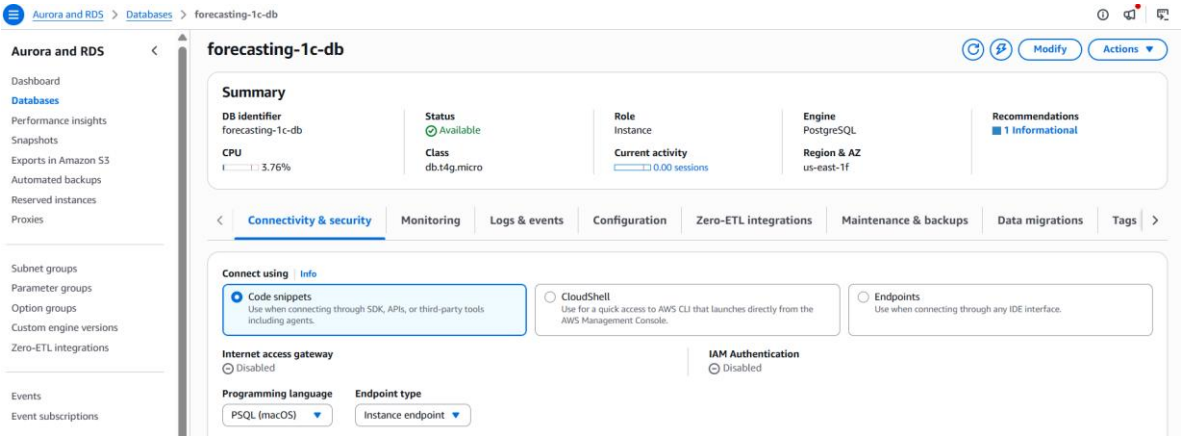
CloudFormation stacks en CREATE_COMPLETE



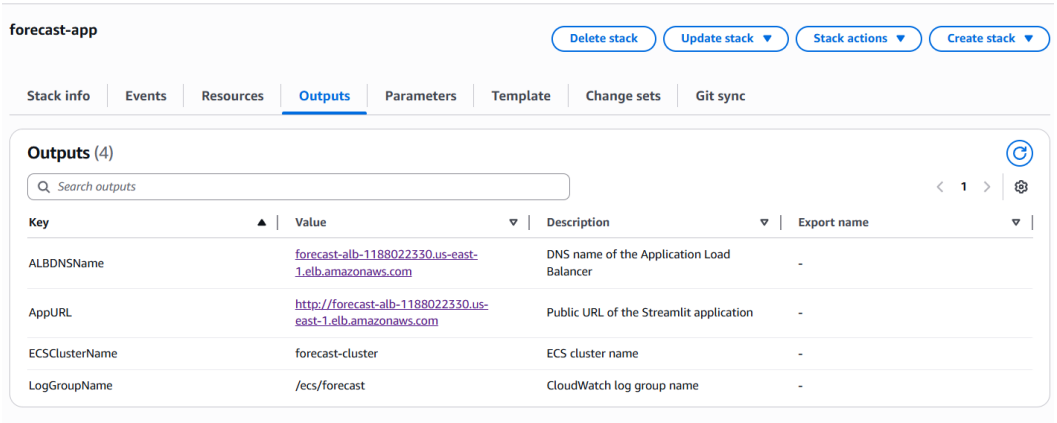
ECR con la imagen publicada

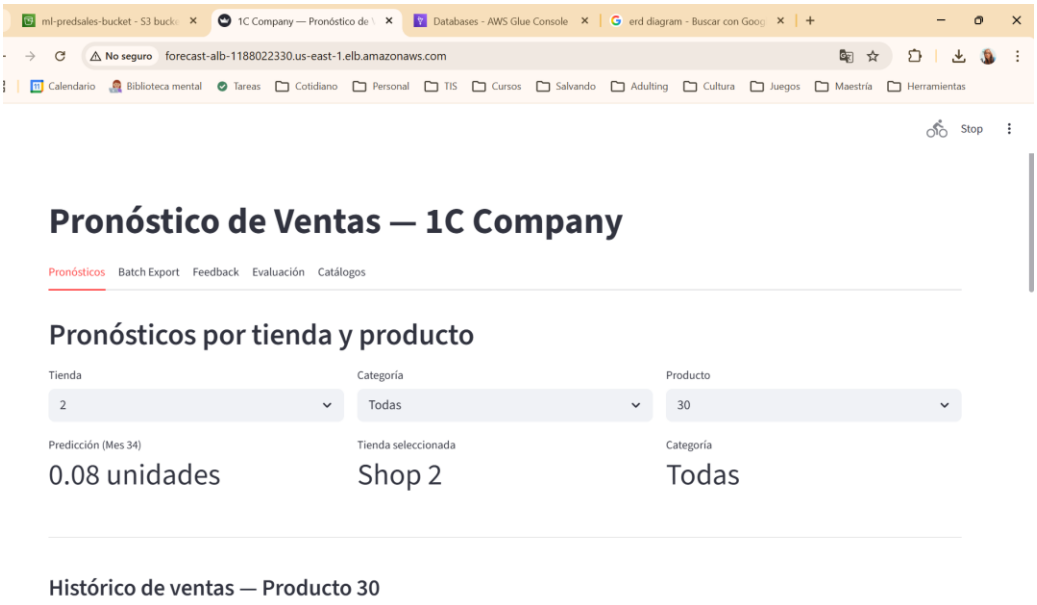


RDS disponible



Endpoint público de app abierto en el browser





Consideraciones de costo y operación

Tenemos 3 servicos que generan costos:

1. **RDS:** se tiene un tamaño t3.micro por lo que el costo aproximado es de $0.017/hora que sería apróximadamente $0.41 al día
2. **ECS Fargate:** dadas las especificaciones se esperaría un costo de $0.01 a la hora, lo que aproximadamente nos daría $0.24 al día
3. **ALB:** este servicio cuesta aproximadamente $0.008/hora, lo que nos daría $0.19 al día
4. **S3:** dado que el volumen no es grande, no creo que genere costos relevantes pero podríamos aproximar $0.001 por hora, $0.02 al día
5. **Glue Crawler:** Cobre cuando corre y dado que no se vuelve a correr, ya no es relevante considerarlo.
6. **ECR:** por el almacenamiento de la imagen, también sería muy bajito, parecido a S3

Lo que sería un costo aproximado de $0.90 al día, $27 al mes. Sin duda un costo mucho menor que tener a un analista corriendo notebooks a manita.

Limitaciones y próximos pasos

Se priorizo más lograr desplegar el streamlit en una URL pública con todos los servicios necesario que fortalecer el modelo. Creo que una gran limitación es que el modelo podría ser mucho más complejo, sin embargo, por cuestiones de tiempo y dado que ya pasaba los requisitos y las validaciones que vimos en clase, corriendo el pipeline completo, confie en su desempeño.

De igual forma, faltó conectar también el modelo para que las predicciones se vayan actualizando con nuevos datos y que pueda ir refinandose poco a poco. Por ahora, toco obtener las predicciones locales y subirlas pero para producción estoy consiente que eso jamás sucedería y que necesitaría tener todo vínculado y automatizado.

De igual forma me gustaría poder jugar más con base de datos para que el streamlit también pueda presentar mucha más información y poder explotar los datos y que eso también esté en el streamlit como adicional.

Uso de herramientas de IA en el proyecto

Siendo honesta, sí utilicé IA para la realización del proyecto, específicamente utilicé Claude.ai para poder debuggear los errores del código. Al intentar utilizar los códigos de las clases o en la creación del streamlit, me salían demasiados errores y en las ocasiones donde no lograba solucionarlo, tenía que consultarlo para que me ayudara a ver dónde estaba el error que no lograba ver.

De igual forma en la creación del stack tuve mucho problemas porque no lograba completarse el servicio ECS y tuve que pedirle que me ayudara a resolverlo. Me revisó el YAML y al final resultó ser un tema de los grupos de seguridad.

Y tambien en los cálculos de los costos, le conté lo que utilicé y todo lo que implica el proyecto y le pedí que me calcular los costos aproximados al mes.

En general, sí lo utilicé como consulta en temas muy específicos de código o cuando me trababa en AWS.